

**LAPORAN PRAKTIKUM 4 TUGAS 1**  
**EKSPERIMEN DENGAN DATA BERBEDA**



**Dosen Pengampu :**

**Triawan Adi Cahyanto M.kom**

**Disusun Oleh :**

**Hafabi Arrohim (2410651013)**

**PROGRAM STUDY TEKNIK INFORMATIKA FAKULTAS**

**TEKNIK**

**UNIVERSITAS MUHAMMADIYAH JEMBER TAHUN 2025**

## **Judul praktikum : Analisis Eksperimen Perbandingan Algoritma**

### **1. Tujuan praktikum**

Mengetahui dan membandingkan performa algoritma penjadwalan CPU (FCFS, SJF, dan Round Robin) berdasarkan waktu tunggu rata-rata, waktu penyelesaian rata-rata, dan jumlah context switching.

### **2. Dasar Teori**

FCFS (First Come First Serve): Proses dilayani sesuai urutan kedatangan. Sederhana, tapi bisa menyebabkan *convoy effect* bila ada proses dengan burst time besar.

SJF (Shortest Job First): Proses dengan burst time terkecil dijalankan lebih dulu. Optimal untuk *average waiting time*, tapi bisa menyebabkan *starvation*.

Round Robin (RR): Setiap proses mendapat jatah waktu (quantum) secara bergantian. Cocok untuk sistem interaktif karena memberikan keadilan bagi semua proses.

### **3. Langkah Percobaan dan Hasil percobaan**

- Buat file program penjadwalan untuk FCFS, SJF, dan RR (dengan quantum = 2, 4, dan 6).

Data set 1 masukkan data :

P1: AT = 0 BT = 10

P2: AT = 1 , BT = 5

P3: AT = 2 BT = 8

```

faby@fabyar-biee:~/TugasOSP4$ gcc sjf.c -o
faby@fabyar-biee:~/TugasOSP4$ ./sjf
=== SIMULASI ALGORITMA SJF (Non-Preemptive) ===
Masukkan jumlah proses: 3

Proses P1
  Arrival Time: 0
  Burst Time: 10

Proses P2
  Arrival Time: 1
  Burst Time: 5

Proses P3
  Arrival Time: 2
  Burst Time: 8

=== HASIL PENJADWALAN SJF ===
PID   AT   BT   CT   TAT   WT
---   --   --   --   ---   --
P1     0    10   10    10    0
P2     1     5   15    14    9
P3     2     8   23    21   13

Average Turnaround Time: 15.00
Average Waiting Time: 7.33

```

Dan liat hasil data set yang sudah di masukkan.

Data set 2 masukkan data :

P1: AT = 0 , BT = 5

P2: AT = 1 , BT = 3

P3: AT = 2 BT = 8

P4: AT = 3 , BT = 6

```

=== SIMULASI ALGORITMA SJF (Non-Preemptive) ===
Masukkan jumlah proses: 4

Proses P1
  Arrival Time: 0
  Burst Time: 5

Proses P2
  Arrival Time: 1
  Burst Time: 3

Proses P3
  Arrival Time: 2
  Burst Time: 8

Proses P4
  Arrival Time: 3
  Burst Time: 6

=== HASIL PENJADWALAN SJF ===
PID   AT   BT   CT   TAT   WT
---   --   --   --   ---   --
P1     0     5     5     5     0
P2     1     3     8     7     4
P3     2     8    22    20    12
P4     3     6    14    11     5

Average Turnaround Time: 10.75
Average Waiting Time: 5.25

```

Dan ini hasil nya dari data set 2 yang sudah di masukkan.

#### 4. Analisis

##### Data Set 1 :

- **FCFS :**

Penjelasan: Proses dikerjakan sesuai urutan kedatangan ( $P1 \rightarrow P2 \rightarrow P3$ ).

Karena P1 memiliki burst time besar, maka P2 dan P3 menunggu lama.

Akibat: Average Waiting Time (WT) tinggi, tapi context switching sedikit.

Kesimpulan: FCFS tidak efisien bila proses pertama sangat panjang.

- **SJF**

Penjelasan: CPU menjalankan proses dengan burst time paling pendek lebih dulu ( $P2 \rightarrow P3 \rightarrow P1$ ).

Proses kecil cepat selesai, sehingga rata-rata WT dan TAT lebih kecil dibanding FCFS.

Kelemahan: Jika proses panjang datang lebih awal, bisa tertunda lama  $\rightarrow$  potensi *starvation*.

Kesimpulan: SJF menghasilkan performa paling baik untuk data ini.

- **Round Robin**

Penjelasan: Semua proses dijalankan secara bergantian sesuai quantum.

Quantum kecil (2): CPU sering berpindah antar proses  $\rightarrow$  context switching tinggi.

Quantum sedang (4): Lebih seimbang antara keadilan & efisiensi.

Quantum besar (6): Mirip FCFS, context switching menurun, tapi fairness berkurang.

Kesimpulan: RR cocok untuk sistem interaktif, bukan untuk efisiensi waktu tunggu.

## **Data Set 2 :**

- **FCFS**

Urutan proses mengikuti waktu kedatangan, tidak memperhatikan burst time.

Karena P3 dan P4 memiliki burst lebih besar, mereka membuat proses berikutnya menunggu lama.

Hasil: Average WT masih cukup tinggi.

- **SJF**

Proses kecil (BT=3, BT=5) diprioritaskan lebih dulu.

Akibat: Waktu tunggu total lebih kecil daripada FCFS.

Namun, proses besar (BT=8) bisa tertunda bila proses kecil terus datang — bisa menimbulkan *starvation*.

Kesimpulan: SJF tetap paling efisien dari segi waktu rata-rata, tapi tidak selalu adil.

- **Round Robin**

Memberi semua proses kesempatan bergantian → paling adil.

Quantum kecil menghasilkan respons cepat tapi overhead tinggi.

Quantum besar menurunkan context switching namun menambah waktu tunggu.

Kesimpulan: RR paling cocok untuk sistem real-time interaktif yang butuh respons cepat, bukan throughput maksimal.

## 5. Kesimpulan Akhir

SJF memberikan hasil *average waiting time* dan *turnaround time* paling kecil.

FCFS paling sederhana, namun tidak efisien untuk proses panjang.

Round Robin paling adil dan cocok untuk sistem interaktif, tetapi context switching tinggi jika quantum terlalu kecil.

Nilai quantum sangat berpengaruh terhadap efisiensi RR.

Tidak ada algoritma yang “sempurna”; pemilihan tergantung jenis sistem dan kebutuhan.

**LAPORAN PRAKTIKUM 4 TUGAS 2**  
**ANALISIS EKSPERIMEN KETIGA ALGORITMA**



**Dosen Pengampu :**

**Triawan Adi Cahyanto M.kom**

**Disusun Oleh :**

**Hafabi Arrohim (2410651013)**

**PROGRAM STUDY TEKNIK INFORMATIKA FAKULTAS**

**TEKNIK**

**UNIVERSITAS MUHAMMADIYAH JEMBER TAHUN 2025**

## Tugas 2

Jawab pertanyaan berikut :

1. Algoritma mana yang memberikan Average Waiting Time terkecil?
2. Mengapa hasil algoritma berbeda-beda?
3. Kapan sebaiknya menggunakan algoritma FCFS?
4. Kapan sebaiknya menggunakan algoritma SJF?
5. Kapan sebaiknya menggunakan algoritma Round Robin?

Jawaban :

1. Algoritma SJF Shortest Job First biasanya memberikan nilai Average Waiting Time terkecil dibanding FCFS dan Round Robin karena selalu memilih process yang paling kecil burst time mula-mula.
2. Akan tetapi, hasil algoritma selalu bervariasi pada tiap kode. Hal ini dikarenakan mekanisme pemilihan proses dan popularitas algoritma yang berbeda-beda, maka urutan eksekusi juga berbeda. Artinya, agak sulit untuk menavigasi masing-masing algoritma dengan kode berbeda. Tidak hanya burst time dari tiap process saja yang berpengaruh, tetapi juga urutan kedatangan dan ukuran dari process quantum ini.
3. FCFS (First-Come, First-Served) sebaiknya digunakan ketika:
  - Sistem sederhana dan jumlah proses sedikit.
  - Tidak ada prioritas khusus, dan seluruh proses diperlakukan sama.
  - Cocok untuk task batch processing dan antrian printing.

#### 4. Algoritma SJF cocok digunakan saat:

- Proses-proses memiliki burst time yang tidak jauh berbeda atau dapat diprediksi.
- Sistem mengutamakan minimasi waktu tunggu rata-rata untuk efisiensi tinggi.
- Sering digunakan di sistem yang memproses job pendek atau kalkulasi cepat.

#### 5. Algoritma Round Robin ideal digunakan ketika:

- Banyak pengguna atau proses yang berjalan secara interaktif (time-sharing system).
- Diperlukan keadilan distribusi CPU time agar semua proses mendapat kesempatan eksekusi.
- Ketika proses memiliki burst time yang bervariasi dan tidak ada prioritas khusus.

**LAPORAN PRAKTIKUM 4 TUGAS 3**  
**MODIFIKASI PROGRAM**



**Dosen Pengampu :**

**Triawan Adi Cahyanto M.kom**

**Disusun Oleh :**

**Hafabi Arrohim (2410651013)**

**PROGRAM STUDY TEKNIK INFORMATIKA FAKULTAS**

**TEKNIK**

**UNIVERSITAS MUHAMMADIYAH JEMBER TAHUN 2025**

## Judul : Memodifikasi Program

pada kode ini tolong tambahkan fitur :

1. tampilkan gantt chart yang lebih detail
2. simpan hasil ke file text
3. baca input dari file

1. Tampilkan gantt chart yang lebih detail

Apa itu gantt chart ??

Gantt Chart digunakan untuk memvisualisasikan alur eksekusi proses di CPU.

Setiap kotak “P1”, “P2”, “P3” menandakan satu proses.

Angka di bawahnya adalah waktu mulai dan selesai eksekusi (dalam satuan waktu CPU).

Ini bentuk program yang saya buat di modifikasi :

```
faby@fabyar-biee:~/TugasOSP4.c$ ./sjf_modifikasi
=== SIMULASI ALGORITMA SJF (Non-Preemptive) ===
Jumlah proses: 3
Data diambil dari file input.txt

=== HASIL PENJADWALAN SJF ===
PID      AT      BT      CT      TAT      WT
---      --      --      --      ---      --
P1        0        7        7        7        0
P2        2        4       12       10        6
P3        4        1        8        4        3

Average Turnaround Time: 7.00
Average Waiting Time: 3.00

=== GANTT CHART ===
| P1 | P3 | P2 |
| 7  | 8  | 12 |
```

Analisis Program :

CPU mulai di waktu 0 → menjalankan P1 selama 7 satuan waktu

Setelah selesai, waktu menjadi 7, lalu CPU menjalankan P2 (burst time = 5 → selesai di waktu 12)

Terakhir, CPU menjalankan P3 selama 1 satuan waktu → selesai di waktu 13

## 2. Simpan hasil ke file teks

Program kamu menggunakan fungsi file handling di bahasa C untuk menulis hasil perhitungan algoritma SJF ke dalam file teks agar bisa disimpan dan dibaca lagi tanpa menjalankan program ulang.

Ini file teks yang saya simpen :

```
faby@fabyar-biee:~/TugasOSP4.c$ cat hasil_sjf.txt
=== HASIL PENJADWALAN SJF ===
PID    AT    BT    CT    TAT    WT
---    --    --    --    ---    --
P1      0     7     7     7     0
P2      2     4    12    10     6
P3      4     1     8     4     3

Average Turnaround Time: 7.00
Average Waiting Time: 3.00

=== GANTT CHART ===
| P1 | P3 | P2 |
0   7   8  12
```

Analisis :

Fitur ini penting karena:

Membuat hasil percobaan tersimpan permanen di file teks, bukan hanya tampil di layar.

Memudahkan kamu mendokumentasikan hasil simulasi untuk laporan tanpa perlu copy manual dari terminal.

File ini bisa kamu gunakan untuk perbandingan hasil antar algoritma CPU Scheduling lain (misalnya FCFS, RR, dll).

Kesimpulan :

Fitur penyimpanan hasil ke file ini menjadikan program lebih efisien dan profesional, karena setiap hasil eksekusi langsung terdokumentasi dalam file teks yang siap dimasukkan ke laporan.

## 3. Baca input dari file

Program kamu tidak lagi meminta input dari pengguna lewat keyboard, tetapi langsung membaca data proses dari file teks **input.txt** menggunakan fungsi file handling di bahasa C

Ini program saya :

```
faby@fabyyar-biee:~/TugasOSP4.c$ cat hasil_sjf.txt
=== HASIL PENJADWALAN SJF ===
PID      AT      BT      CT      TAT      WT
---      --      --      --      ---      --
P1        0        7        7        7        0
P2        2        4       12       10        6
P3        4        1        8        4        3

Average Turnaround Time: 7.00
Average Waiting Time: 3.00

=== GANTT CHART ===
| P1 | P3 | P2 |
0    7    8   12
```

```
faby@fabyyar-biee:~/TugasOSP4.c$ ls
hasil_sjf.txt  input.txt  sjf_modifikasi  sjf_modifikasi.c
faby@fabyyar-biee:~/TugasOSP4.c$ |
```

Fitur membaca input dari file memberikan beberapa keuntungan:

Lebih efisien: tidak perlu mengetik data proses setiap kali menjalankan program.

Lebih konsisten: kamu bisa menyimpan beberapa file input berbeda untuk uji coba (misal input1.txt, input2.txt).

Mudah diuji ulang: cukup ganti isi file tanpa mengubah kode program.



**LAPORAN PRAKTIKUM 4 TUGAS 4**  
**ANALISIS EKSPERIMEN PERBANDINGAN ALGORITMA**



**Dosen Pengampu :**  
**Triawan Adi Cahyanto M.kom**

**Disusun oleh:**  
**Hafabi Arrohim (2410651013)**

**PROGRAM STUDY TEKNIK INFORMATIKA**  
**FAKULTAS TEKNIK**  
**UNIVERSITAS MUHAMMADIYAH JEMBER**  
**TAHUN 2025**

## Judul : Perbandingan Algoritma Penjadwalan

### 1. Test Case 1

Jumlah proses: 4

Time quantum: 3

P1: AT = 0 BT = 24

P2: AT = 1 BT = 3

P3: AT = 2 , BT = 3

P4: AT = 3 , BT = 6

Pada kasus ini burst time setiap proses berbeda-beda, ada yang lama dan ada yang cepat :

FCFS menghasilkan waktu tunggu tinggi karena proses pertama (P1) sangat lama sehingga proses lain menunggu lama.

SJF lebih baik karena proses dengan waktu pendek dikerjakan lebih dulu sehingga waktu tunggu rata-rata lebih kecil.

Round Robin ( $q=3$ ) lebih adil karena setiap proses mendapat giliran, tetapi banyak *context switching* sehingga efisiensinya sedikit berkurang.

Kesimpulan: SJF paling efisien untuk kasus ini, RR paling adil, FCFS paling lambat.

Masukkan data nya kemudian akan keluar hasil seperti ini :

PROGRAM PERBANDINGAN ALGORITMA CPU SCHEDULING

Program ini akan membandingkan 3 algoritma:

1. FCFS (First Come First Served)
2. SJF (Shortest Job First)
3. Round Robin

Masukkan jumlah proses: 4

Masukkan time quantum untuk Round Robin: 3

== Proses P1 ==

Arrival Time: 0

Burst Time: 24

== Proses P2 ==

Arrival Time: 1

Burst Time: 3

== Proses P3 ==

Arrival Time: 2

Burst Time: 3

== Proses P4 ==

Arrival Time: 3

Burst Time: 6

MENJALANKAN ALGORITMA FCFS

MENJALANKAN ALGORITMA FCFS

HASIL ALGORITMA FCFS

| PID | AT | BT | CT | TAT | WT |
|-----|----|----|----|-----|----|
| P1  | 0  | 24 | 24 | 24  | 0  |
| P2  | 1  | 3  | 27 | 26  | 23 |
| P3  | 2  | 3  | 30 | 28  | 25 |
| P4  | 3  | 6  | 36 | 33  | 27 |

Average Turnaround Time: 27.75

Average Waiting Time: 18.75

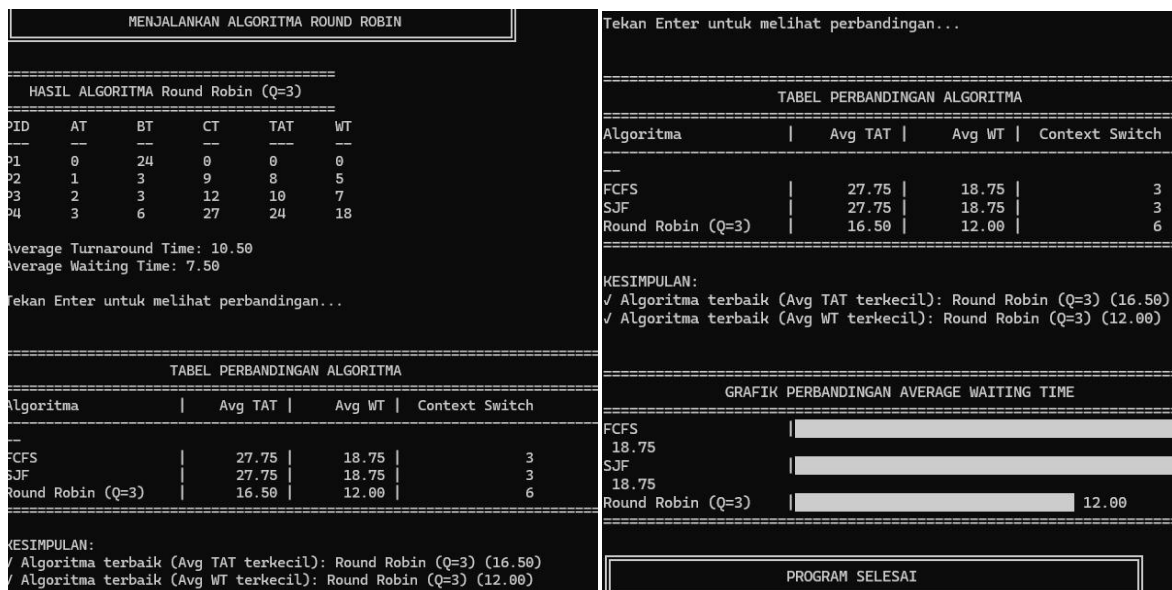
Tekan Enter untuk melanjutkan...

MENJALANKAN ALGORITMA SJF

HASIL ALGORITMA SJF

| PID | AT | BT | CT | TAT | WT |
|-----|----|----|----|-----|----|
| P1  | 0  | 24 | 24 | 24  | 0  |
| P2  | 1  | 3  | 27 | 26  | 23 |
| P3  | 2  | 3  | 30 | 28  | 25 |
| P4  | 3  | 6  | 36 | 33  | 27 |

Average Turnaround Time: 27.75



## 2. Test Case 2

Jumlah proses: 5

Time quantum: 4

P1: AT = 0 , BT = 10

P2: AT = 0 , BT = 5

P3: AT = 0 , BT = 8

-P4: AT = 0 , BT = 12

-P5: AT = 0 , BT = 6

Semua proses tiba di waktu yang sama (AT=0).

FCFS tergantung urutan input, jika proses panjang dikerjakan dulu maka waktu tunggu meningkat

SJF paling efisien karena bisa langsung memilih proses dengan burst time paling kecil terlebih dahulu.

Round Robin (q=4) memberikan hasil yang cukup seimbang antara efisiensi dan keadilan.

Kesimpulan: SJF terbaik pada kasus ini karena semua proses sudah siap sejak awal.

Masukkan data nya kemudian lihat hasilnya :

```
PROGRAM PERBANDINGAN ALGORITMA CPU SCHEDULING

Program ini akan membandingkan 3 algoritma:
1. FCFS (First Come First Served)
2. SJF (Shortest Job First)
3. Round Robin

Masukkan jumlah proses: 5
Masukkan time quantum untuk Round Robin: 4

== Proses P1 ==
Arrival Time: 0
Burst Time: 10

== Proses P2 ==
Arrival Time: 0
Burst Time: 5

== Proses P3 ==
Arrival Time: 0
Burst Time: 8

== Proses P4 ==
Arrival Time: 0
Burst Time: 12

== Proses P5 ==
Arrival Time: 0
Burst Time: 6
```

| MENJALANKAN ALGORITMA FCFS       |    |    |    |     |    |
|----------------------------------|----|----|----|-----|----|
| HASIL ALGORITMA FCFS             |    |    |    |     |    |
| PID                              | AT | BT | CT | TAT | WT |
| P1                               | 0  | 10 | 10 | 10  | 0  |
| P2                               | 0  | 5  | 15 | 15  | 10 |
| P3                               | 0  | 8  | 23 | 23  | 15 |
| P4                               | 0  | 12 | 35 | 35  | 23 |
| P5                               | 0  | 6  | 41 | 41  | 35 |
| Average Turnaround Time: 24.80   |    |    |    |     |    |
| Average Waiting Time: 16.60      |    |    |    |     |    |
| Tekan Enter untuk melanjutkan... |    |    |    |     |    |
| MENJALANKAN ALGORITMA SJF        |    |    |    |     |    |
| HASIL ALGORITMA SJF              |    |    |    |     |    |
| PID                              | AT | BT | CT | TAT | WT |
| P1                               | 0  | 10 | 29 | 29  | 19 |
| P2                               | 0  | 5  | 5  | 5   | 0  |
| P3                               | 0  | 8  | 19 | 19  | 11 |

### 3. Test case 3

Jumlah proses: 5

Time quantum: 2

P1: AT = 0 , BT = 8

P2: AT = 2 BT = 4

P3: AT = 4 BT = 9

P4: AT = 6 BT = 5

P5: AT = 8 BT = 3

Proses datang tidak bersamaan (selang waktu berbeda).

FCFS membuat proses baru menunggu lama jika proses sebelumnya belum selesai.

SJF tetap efisien, tapi karena proses datang bergantian hasilnya tidak seoptimal kasus kedua.

Round Robin ( $q=2$ ) lebih responsif karena setiap proses dapat giliran lebih cepat, walaupun terjadi lebih banyak *context switching*.

Kesimpulan: RR paling cocok untuk kondisi proses datang bergantian karena lebih responsif, meski sedikit boros waktu CPU.

```

PROGRAM PERBANDINGAN ALGORITMA CPU SCHEDULING

Program ini akan membandingkan 3 algoritma:
1. FCFS (First Come First Served)
2. SJF (Shortest Job First)
3. Round Robin

Masukkan jumlah proses: 5
Masukkan time quantum untuk Round Robin: 2

== Proses P1 ==
Arrival Time: 0
Burst Time: 8

== Proses P2 ==
Arrival Time: 2
Burst Time: 4

== Proses P3 ==
Arrival Time: 4
Burst Time: 9

== Proses P4 ==
Arrival Time: 6
Burst Time: 5

== Proses P5 ==
Arrival Time: 8
Burst Time: 3

```

```
=====
HASIL ALGORITMA SJF
=====
PID      AT      BT      CT      TAT      WT
-----
P1        0        8        8        8        0
P2        2        4        15       13        9
P3        4        9        29       25       16
P4        6        5        20       14        9
P5        8        3        11        3        0

Average Turnaround Time: 12.60
Average Waiting Time: 6.80

Tekan Enter untuk melanjutkan...
```

```

=====
MENJALANKAN ALGORITMA FCFS
=====
HASIL ALGORITMA FCFS
=====

```

| PID | AT | BT | CT | TAT | WT |
|-----|----|----|----|-----|----|
| P1  | 0  | 8  | 8  | 8   | 0  |
| P2  | 2  | 4  | 12 | 10  | 6  |
| P3  | 4  | 9  | 21 | 17  | 8  |
| P4  | 6  | 5  | 26 | 20  | 15 |
| P5  | 8  | 3  | 29 | 21  | 18 |

```

Average Turnaround Time: 15.20
Average Waiting Time: 9.40

Tekan Enter untuk melanjutkan...

```

| MENJALANKAN ALGORITMA SJF |    |    |    |     |    |
|---------------------------|----|----|----|-----|----|
| =====                     |    |    |    |     |    |
| HASIL ALGORITMA SJF       |    |    |    |     |    |
| =====                     |    |    |    |     |    |
| PID                       | AT | BT | CT | TAT | WT |
| P1                        | 0  | 8  | 8  | 8   | 0  |
| P2                        | 2  | 4  | 15 | 13  | 9  |
| P3                        | 4  | 9  | 29 | 25  | 16 |

| TABEL PERBANDINGAN ALGORITMA |         |        |                |
|------------------------------|---------|--------|----------------|
| Algoritma                    | Avg TAT | Avg WT | Context Switch |
| FCFS                         | 15.20   | 9.40   | 4              |
| SJF                          | 12.60   | 6.80   | 4              |
| Round Robin (Q=2)            | 12.80   | 6.40   | 6              |

KESIMPULAN:  
 ✓ Algoritma terbaik (Avg TAT terkecil): SJF (12.60)  
 ✓ Algoritma terbaik (Avg WT terkecil): Round Robin (Q=2) (6.40)



PROGRAM SELESAI



**LAPORAN PRAKTIKUM 4 TUGAS 5**  
**ANALISIS EKSPERIMEN PERBANDINGAN ALGORITMA**



**Dosen Pengampu :**  
**Triawan Adi Cahyanto M.kom**

**Disusun Oleh :**  
**Hafabi Arrohim (2410651013)**

**PROGRAM STUDY TEKNIK INFORMATIKA FAKULTAS**  
**TEKNIK**  
**UNIVERSITAS MUHAMMADIYAH JEMBER TAHUN 2025**

## **1. Analisis performa**

### **a. Dalam kondisi apa fcfs memberikan hasil terbaik?**

FCFS (First Come First Serve) memberikan hasil terbaik ketika semua proses datang secara berurutan (arrival time berdekatan) dan memiliki burst time hampir sama. Dalam kondisi ini, tidak ada proses yang menunggu lama di belakang proses panjang, sehingga waktu tunggu (waiting time) relatif kecil.

### **b. Mengapa SJF hampir selalu memberikan average WT terkecil ?**

SJF (Shortest Job First) memprioritaskan proses dengan waktu eksekusi paling pendek terlebih dahulu. Karena proses kecil selesai lebih cepat, total waktu tunggu rata-rata menjadi lebih kecil. Secara teori, SJF memang terbukti optimal dalam meminimalkan average waiting time.

### **c. Apa trade-off dari Round Robin ?**

Round Robin (RR) memberikan keadilan (fairness) karena setiap proses mendapat giliran, tetapi banyak context switching terjadi, terutama jika quantum-nya terlalu kecil. Akibatnya, overhead meningkat dan waktu eksekusi total (turnaround time) bisa lebih besar.

## **2. Context Switching**

### **a. Algoritma mana yang paling banyak melakukan context switching?**

Round Robin, karena CPU sering berpindah antar proses setiap kali quantum habis

**b. Bagaimana pengaruh quantum terhadap jumlah context switching di round robin ?**

Semakin kecil quantum, semakin banyak context switching yang terjadi. Sebaliknya, jika quantum terlalu besar, Round Robin akan semakin mirip FCFS.

**c. Coba jalankan round robin dengan quantum 2,4,6 – apa yang terjadi ?**

Quantum 2: sangat sering berganti proses → context switching tinggi → overhead besar.

Quantum 4: lebih seimbang antara fairness dan efisiensi.

Quantum 6: mirip FCFS, proses panjang mendominasi, fairness menurun.

**3. Fairness (Keadilan)**

**a. Algoritma mana yang paling “adil”?**

Round robin yang paling adil karena setiap proses mendapat jatah CPU secara bergantian.

**b. Proses mana yang paling diuntungkan di algoritma FCFS?**

Proses yang lebih awal (arrival time kecil) diuntungkan, karena langsung dieksekusi tanpa menunggu lama.

**c. Apakah ada kemungkinan starvation di SJF?**

Ya jika banyak proses pendek datang terus – menerus. Maka proses yang Panjang tidak pernah mendapat giliran, sehingga terjadi starvation.

#### **4. Real world application**

**a. Untuk web server yang melayani banyak request kecil, algoritma mana yang cocok?**

SJF atau Shortest Remaining Time First (SRTF), karena request kecil bisa diselesaikan cepat dan meningkatkan throughput.

**b. Untuk render video (task besar), algoritma mana yang cocok?**

FCFS, karena setiap proses panjang dapat berjalan penuh tanpa interupsi — cocok untuk pekerjaan batch yang tidak butuh interaksi pengguna.

**c. Untuk OS desktop (interaktif), algoritma mana yang cocok?**

Round Robin dengan quantum sedang (misalnya 4–8 ms), karena memberikan respons cepat ke pengguna tanpa mengorbankan keadilan.